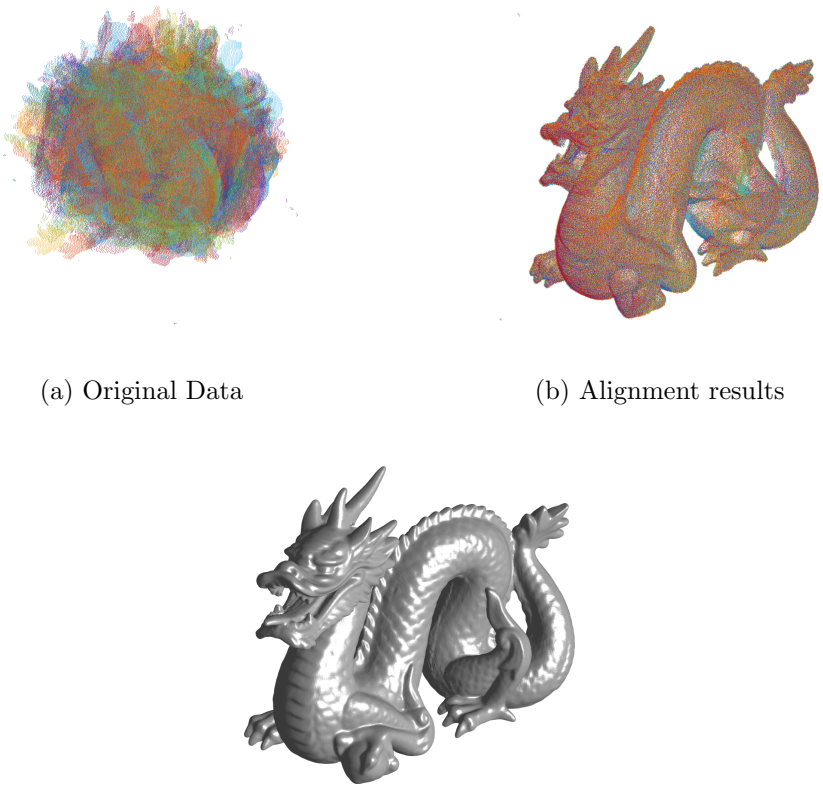
Automatic 3D Registration Toolkit
User Guide

Introduction

Automatic 3D Registration Toolkit is a MATLAB [1] toolbox that allows the reconstruction of 3D models starting from their partial views (Figure 1). The toolkit is implemented as MATLAB and C++ code (MEX files) using the libraries OpenMesh [2], Eigen [3], ANN [4] and CVLab [5]. ART can be downloaded for free at the site `http://profs.sci.univr.it/~castella/Sources.html`.

The datasets used in this paper are provided by the *Stanford Computer Graphics Laboratory* (`http://graphics.stanford.edu/data/3Dscanrep/`).



(a) Original Data

(b) Alignment results

(c) Reconstructed model

Figure 1: Automatic 3D Registration Toolkit.

Installing ART

As a first step, the zip file `art.zip` must be extracted in a folder of choice (for our examples the `...\Documents\MATLAB` folder is used). Next, we add the toolkit to MATLAB's search path. To do this, we type in the MATLAB command line:

```
addpath ...\Documents\MATLAB\art;                % Selected path
addpath ...\Documents\MATLAB\art\gui;
addpath ...\Documents\MATLAB\art\plyio;
addpath ...\Documents\MATLAB\art\poisson;
savepath;
```

to complete the toolkit installation process.

Uninstalling ART

The ART uninstall process is similar to the one described in Installing ART. We write in the MATLAB command line:

```
rmpath ...\Documenti\MATLAB\art;    % Installation path
rmpath ...\Documenti\MATLAB\art\gui;
rmpath ...\Documenti\MATLAB\art\plyio;
rmpath ...\Documenti\MATLAB\art\poisson;
savepath;
```

to remove the toolbox from MATLAB's search path. Then, we can type

```
rmdir(...\Documenti\MATLAB\art, 's');
```

to delete the (`...\Documenti\MATLAB\art`) folder from the hard drive.

Main window

Once the installation process is complete (see Installing ART), the ART GUI (Figure 2) can be called by writing in the MATLAB's command line the directive:

```
[data, settings] = artgui;
```

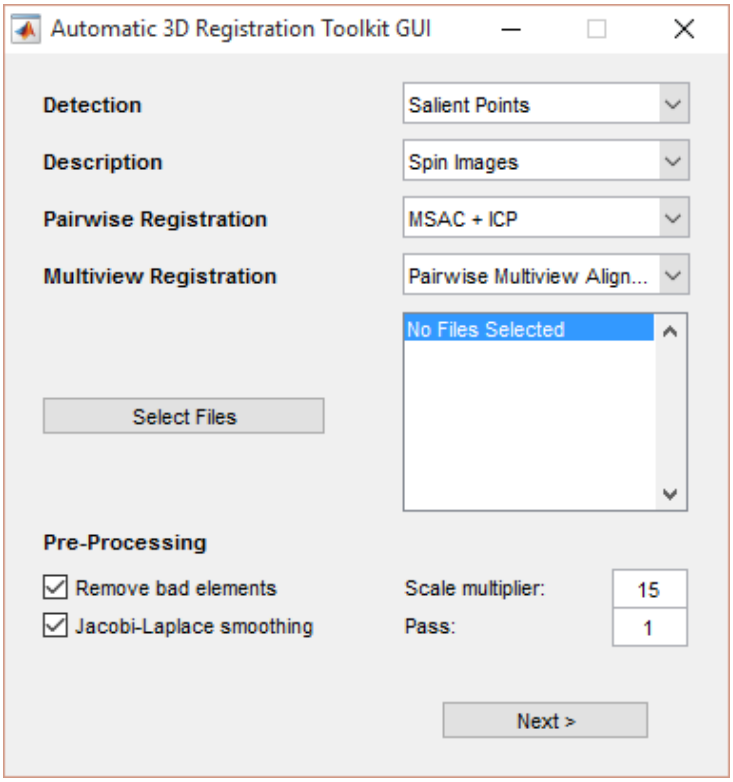


Figure 2: ART GUI.

This command produces the output data structures called *data* and *settings*. The structure *data* stores all the data relative to the toolkit computation (such as the vertex and the faces of the meshes, the keypoints and their descriptions, the pairwise and global rigid transformation matrices) while the *settings* structure stores all the parameters needed by the tool (which modules are used and the parameters necessary to their functioning).

In the upper side of the window, we can choose the algorithms relative to each module of the pipeline:

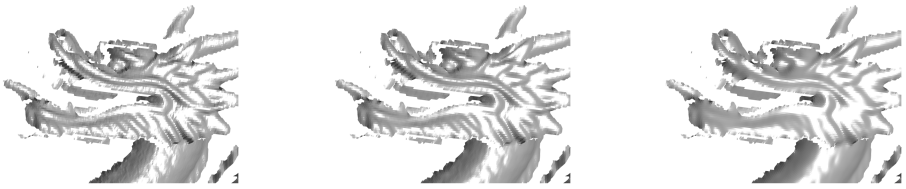
- **Detection**: from this drop-down menu we can choose the keypoint extraction module.
- **Description**: from this drop-down menu we can choose the keypoint description module.
- **Pairwise Registration**: from this drop-down menu we can choose the pairwise registration module.
- **Multiview Registration**: from this drop-down menu we can choose the multi-view registration module.

In the middle of the window, we can select the mesh files composing the model to reconstruct. Note that the input meshes must be triangular (as in, they are composed only by triangular faces).

In the lower side of the window the parameters relative to the view pre-processing module can be set:

- **Remove bad elements**: here we decide if the bad elements removal step will be performed. If we decide to remove the bad elements, the **Scale multiplier** value (**default: 15**) equals to the scale of the X84 statistic [6, 7]. If we increase this value, the process will remove less elements.
- **Jacobi-Laplace smoothing**: here we decide if the initial mesh smoothing will be performed. If we decide to perform this process, the **Pass** value (**default**: 1) sets the number of smoothing passages performed (Figure 3). We choose this value based on the acquisition noise amount; if it is small we can perform few or no smoothing passages, on the other hand, if it is high we have to perform more passages. Note that being too aggressive in this step can remove some of the finer details of the model.

Once all the parameters are set, we can click on the *Next* button to open the keypoint extraction window.



(a) Smooth 0 pass. (b) Smooth 1 pass. (c) Smooth 5 pass.

Figure 3: The pre-processing smoothing applied to the view `dragon-stand-000.ply`.
Performing too many passages can remove some of the finer details of the mesh.

Keypoint extraction window

The keypoint extraction window varies depending on the module selected in Main window. At this moment, only the Salient Points [8] module is available.

This module generates for each view a vector containing the keypoint vertex indices and stores them into the data structure *data.keypoints*.

Salient Points window

If we choose the Salient Points module we will be presented with the window shown in Figure 4. From this window, we can modify the main parameters of the algorithm:

- **Boundary inhibition (General)**: here we select the parameter relative to the mesh boundary inhibition (**default**: 3), This parameter is defined as a multiple of the average mesh resolution. Varying this parameter affects the amount of surface near the boundary of the mesh that won't be considered during the salient points computation.
- **Base sigma (Gaussians)**: here we decide the ϵ value used during the gaussian filters computation (**defautl**: 0.25). This parameter is defined as a function of the average resolution. Increasing this value increases the distribution of the keypoints (Figure 5) at the cost of computational time.
- **Range (Gaussians)**: here we select the parameter relative to the

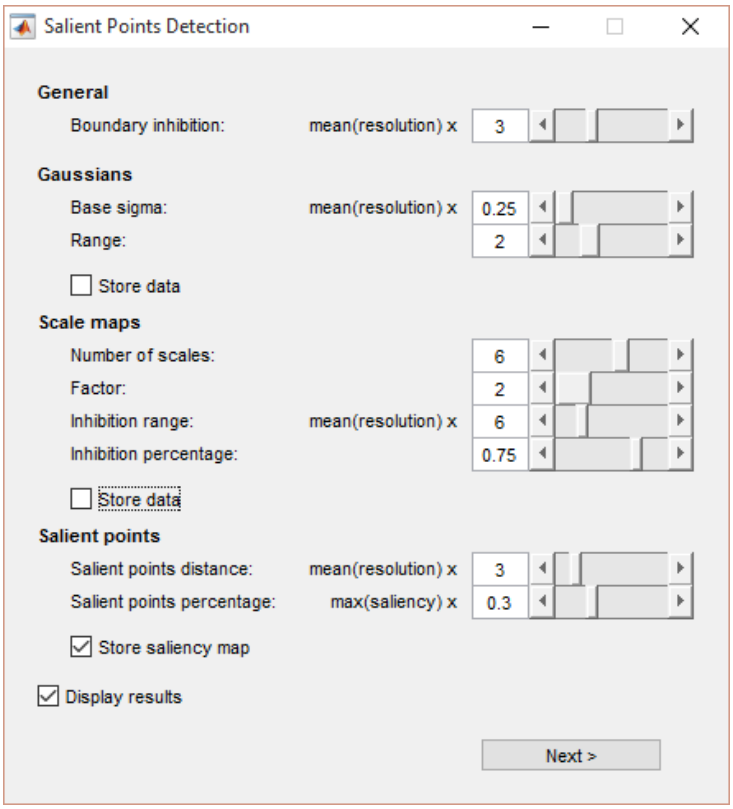
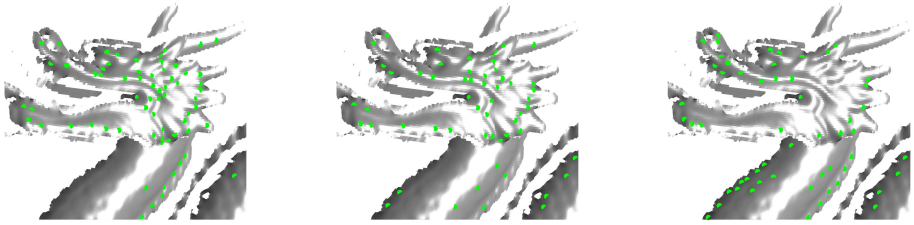


Figure 4: The Salient Points window.

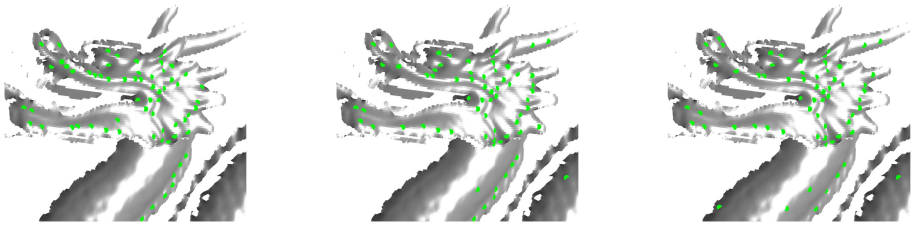


(a) $\epsilon = 0.25\overline{res}$ (b) $\epsilon = 0.50\overline{res}$ (c) $\epsilon = 1.00\overline{res}$

Figure 5: The salient points extracted from the view `dragon-stand-000.ply` for different values of ϵ.

radius of the neighborhood considered by the gaussian filters (**default**: 2). Increasing this parameter slightly improves the precision of the filtering and increases the computational time.

- **Number of scales (Scale maps)**: here we decide the number N of scale maps to be computed (**default**: 6). Incrementing this value increases the distribution of the keypoints (Figure 6) and the time needed by the algorithm.

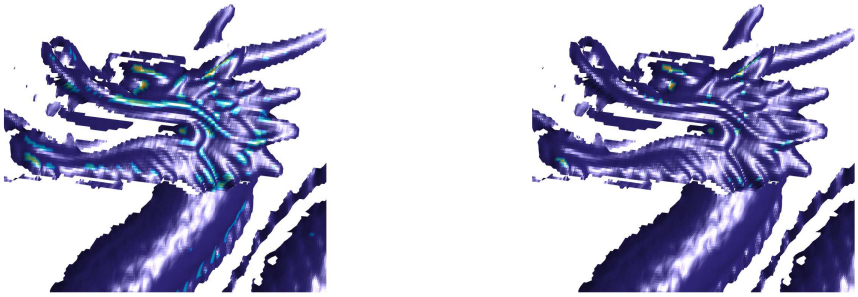


(a) $N = 4$ (b) $N = 6$ (c) $N = 8$

Figure 6: The salient points extracted from te view `dragon-stand-000.ply` for different values of N.

- **Factor (Scale maps)**: here we select the parameter k used by the DoG operator (**default**: 2). Increasing this value increases the size of the areas considered by each scale map at the cost of computational time.
- **Inhibition range (Scale maps)**: here we decide the radius of the neighborhood considered during the scale map inhibition process (**default**: 6). Increasing this radius reduces the total number of extracted keypoints, but increases their importance.
- **Inhibition percentage (Scale maps)**: here we select the parameters relative to the percentage of neighbors that have to be lower than the considered point during the scale maps inhibition process (**default**: 0.75). Increasing this value reduces the number of extracted keypoints, which will be more significant.
- **Salient points distance (Salient points)**: here we decide the minimum distance allowed between the different keypoints extracted (**default**: 3). This parameter is defined as a multiple of the mean mesh resolution. Increasing this value reduces the number of extracted keypoints.

- **Salient points percentage (Salient points)**: here we select the percentage p_supp of the global maximum of the saliency map used in the non maxima suppression phase (**default**: 0.3). Increasing this parameter reduces the number of extracted keypoints (Figure 7).



(a) $p_{supp} = 0.3max(S)$ (b) $p_{supp} = 0.6max(S)$

Figure 7: The saliency map of the view `dragon-stand-000.ply` for different values of p_{supp}.

We can tick the check boxes of the window to select which features we want to store into the *data* structure and if we want to visualize the extracted keypoints information. Note that only the selected features will be visualized.

Once all the parameters have been set, we can click on the *Next* button to open the keypoint description window.

Keypoint description window

The keypoint description window varies depending on the module selected in Main window. At this moment, only the Spin Images [9] descriptor is available.

This module generates, for each keypoint calculated by the extraction module, a vector containing the description of the surface located near the considered keypoint. For each view, the result is a matrix that is stored into the *data.descriptors* structure.

Spin Images window

If we choose the Spin Images module we will be presented with the window shown in Figure 8, which allows us to set the main parameters of the algorithm:

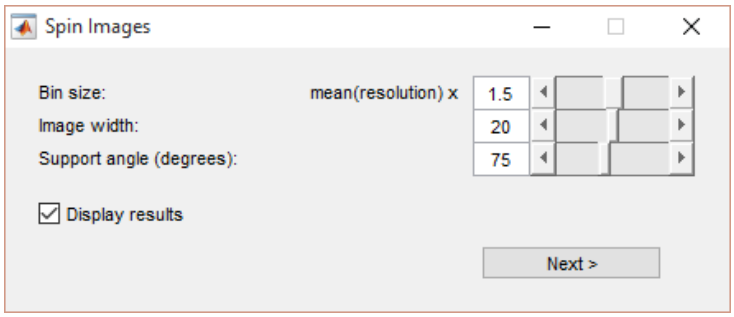
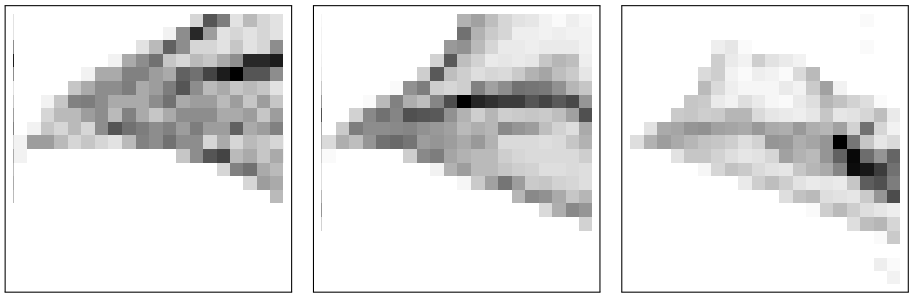


Figure 8: The Spin Image window.

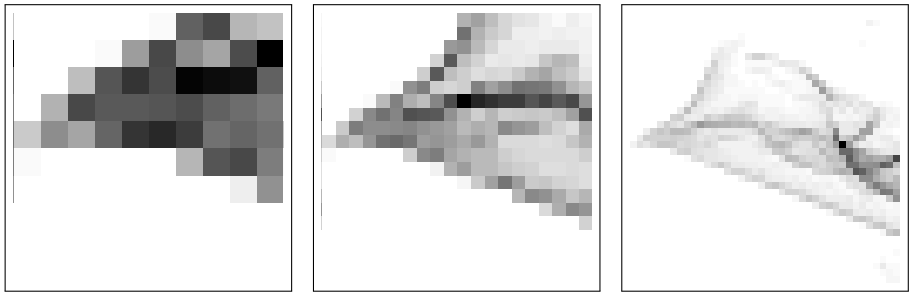
- **Bin size**: here we set the bin size b of the spin image (**default**: 1.5), which is defined as multiple of the mean mesh resolution. Increasing this value makes every bin of the spin image less descriptive and increments the size of the neighborhood considered by the descriptor (Figure 9).



(a) $b = 0.75\overline{res}$ (b) $b = 1.5\overline{res}$ (c) $b = 3\overline{res}$

Figure 9: A spin image extracted from the view `dragon-stand-000.ply` for different values of b.

- **Image width**: here we decide the size of the window W considered during the computation of the descriptor (**default**: 20). Increasing this value increases the total number of bins composing the spin image without affecting the information stored in each one of them, therefore we consider a larger area of the surface for each descriptor (Figure 10).



(a) $W = 10$ (b) $W = 20$ (c) $W = 40$

Figure 10: A spin image extracted from the view `dragon-stand-000.ply` for different values of W.

- **Support angle**: here we select the support angle A_s (**default**: 75°). This value sets the maximum angle allowed between the normals of the different vertices that compose the spin image. This parameter doesn't really affect the final results as long as we consider the partial views of the model.

By ticking the **Display results** check box we decide if we want to show the results of the keypoint description.

Once all the parameters have been set, we can click on the *Next* button to open the pairwise registration window.

Pairwise Registration window

The pairwise registration window varies depending on the module selected in Main window. At this moment, only the MSAC + ICP module is available.

This module executes different tasks. Initially, it approximatively determinates which of the view pairs composing the model overlap. Then, it calculates the corresponding keypoints between those view pairs. Finally, it determinates the rigid transformation matrices that align them. The result of this module is a matrix of rigid transformation matrices which is stored into the *data.pairwise_transforms* structure.

MSAC + ICP window

If we choose the MSAC + ICP module we will be presented with the window shown in Figure 11, which allows us to set the main parameters of the algorithm:

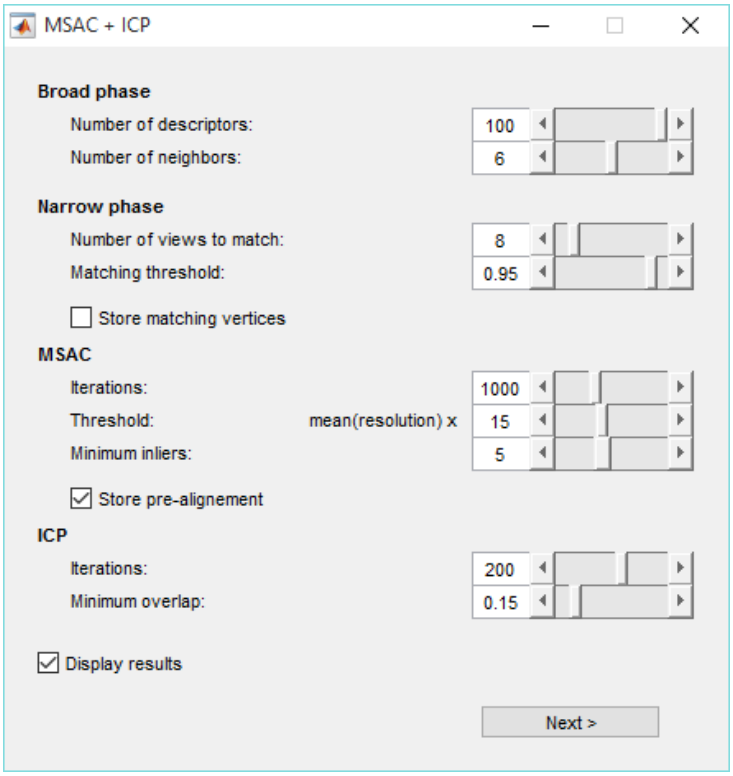
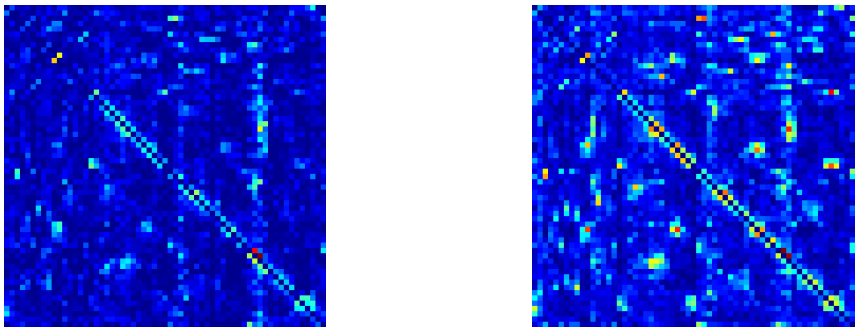


Figure 11: The MSAC + ICP window.

- **Number of descriptors (Broad phase)**: here we select the number of descriptors n_d to be considered during the broad phase of the algorithm (**default**: 100). Decreasing this value reduces the precision of the co-occurances matrix (Figure 12).
- **Number of neighbors (Broad phase)**: here we decide the number of similar descriptors n_n to be considered during the broad phase (**default**: 6). Increasing this value increments the precision of the co-occurances matrix.
- **Number of views to match (Narrow phase)**: here we set the number n_v of views to match for each view (**default**: 8). Increasing this value increases the number of pairs of views compared.



(a) $n_d = 50$ (b) $n_d = 100$

Figure 12: The co-occurances matrix of the dragon model for different values of n_d.

- **Matching threshold (Narrow phase)**: here we set the value of the ratio v_r between the distance of the 1-nn and 2-nn descriptors used during the computation of the matching vertices (**default**: 0.95). Increasing this value increments the number of correspondences found, but increases the chance of error.
- **Iterations (MSAC)**: here we decide the maximum number of iteration m_i allowed to the MSAC algorithm (**default**: 1000). Increasing this value increments the number of combinations of correspondences tested by the algorithm.
- **Threshold (MSAC)**: here we decide the radius m_t within which each point will be considered inlier by the MSAC algorithm (**default**: 15). This parameter is defined as a multiple of the average mesh resolution. Increasing this value produces more, less accurate, pre-alignment rigid transformation matrices.
- **Minimum inliers (MSAC)**: here we set the minimum number m_n of inliers allowed by MSAC (**default**: 5). Increasing this value increments the chance that the pre-alignment matrix calculated by MSAC is valid.
- **Iterations (ICP)**: here we decide the maximum number of iteration i_i allowed to the ICP algorithm (**default**: 200). Increasing this value increments the number of computed alignments.
- **Minimum overlap (ICP)**: here we set the size i_o of the minimum

overlapping area needed to validate the ICP alignment (**default**: 15).

We can tick the check boxes of the window to select which features we want to visualize and store into the *data* structure.

Once all the parameters have been set, we can click on the *Next* button to open the multi-view registration window.

Multi-view Registration window

The multi-view registration window varies depending on the module selected in Main window. At this moment, only the Pairwise Multiview Alignment is available.

This module chains the pairwise rigid transformation matrices in order to align all the views to a reference. In this way. we get the global registration needed to reconstruct the scanned model. This information is then stored into the *data.rigid_transform* structure.

Pairwise Multiview Alignment window

If we choose the Pairwise Multiview Alignment module we will be presented with the window shown in Figure 8, which allows us to set the main parameters of the algorithm:

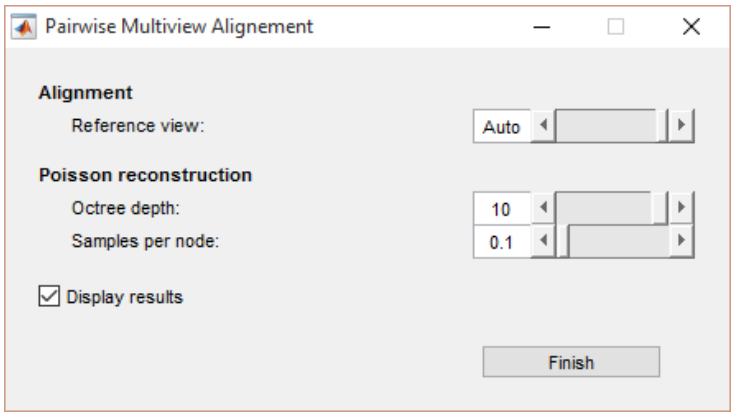
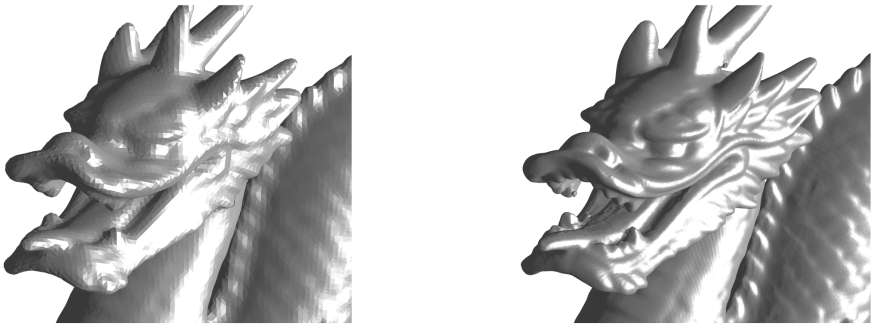


Figure 13: The Pairwise Multiview Alignment window.

- **Reference view (Alignment)** here we choose the index of the reference view (**default**: Auto). The value *Auto* sets as the reference one of the views that aligns with the highest number of views.

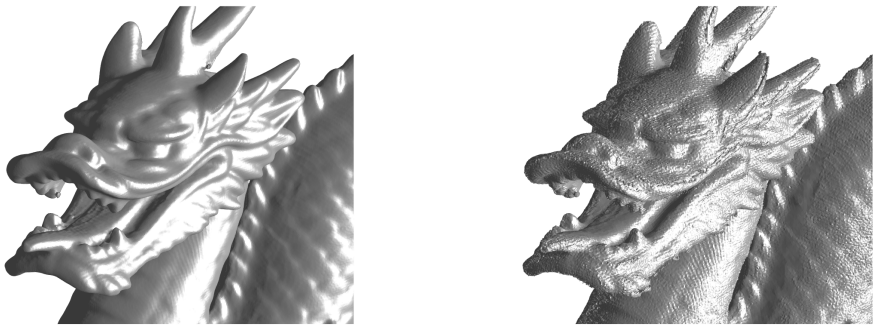
- **Octree depth (Poisson reconstruction)**: here we set the depth p_d of the tree used in the surface reconstruction process (**default**: 10). Decreasing this value reduces the precision of the process and the size of the output (Figure 14).



(a) $p_d = 8$ (b) $p_d = 10$

Figure 14: The reconstructed model for different values of p_d.

- **Samples per node (Poisson reconstruction)**: here we set the number of samples p_s used for each node of the reconstruction tree (**default**: 0.1). Increasing this value increments the precision of the process (Figure 15).



(a) $p_s = 0.1$ (b) $p_s = 5$

Figure 15: The reconstructed model for different values of p_s.

The last parameters refer to the algorithm *Poisson Surface Reconstruction* [10, 11], used as the final step of the module in order to reconstruct the faces that compose the reconstructed model.

By ticking the **Display results** check box we decide if we want to show the results of the global alignment.

Once all the parameters have been set, we can click on the *Next* button to proceed with the execution of ART.

Expanding ART

In this section, we describe the process of adding new modules to the ART toolkit.

Adding a Keypoint Extraction module

Let's assume we want to add a new keypoint extraction module called **customextraction** to ART.

As a first step we need to add the new module to the drop-down menu *Detection* in the main window of the toolkit (Figure 2). We digit

```
guide artgui;
```

in the MATLAB command line to modify ART's main window (Figure 16)

Then, we right-click on the drop-down menu and select *Property Inspector*. In the following window we edit the field *String* adding the new element **Custom Extraction**.

After that, we edit the GUI's code using the command

```
edit artgui;
```

to modify the functions *artgui_OpeningFcn* and *detection_popup_Callback*:

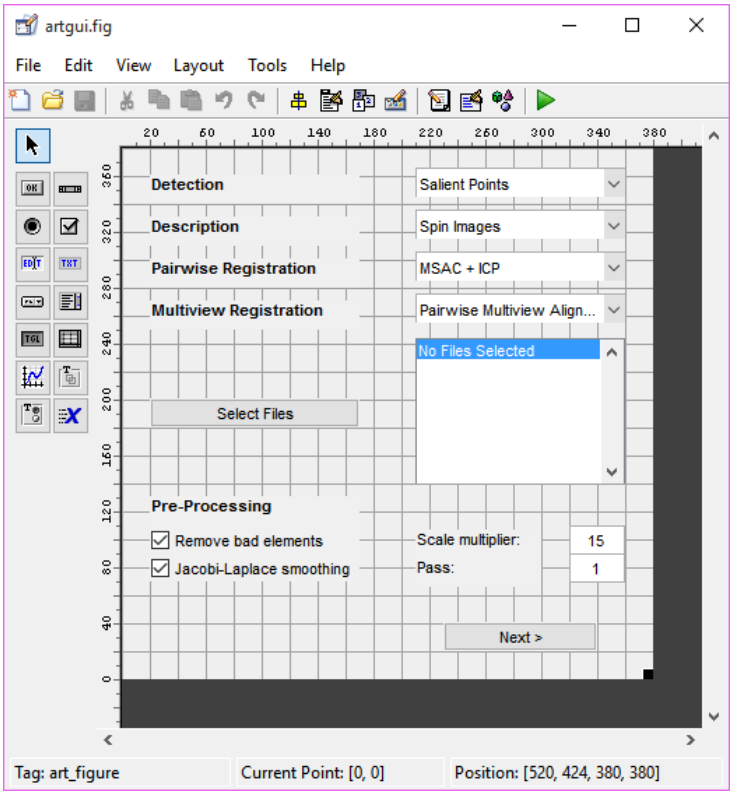


Figure 16: The editing window relative to ART's GUI.

```
...
% Get the detection (add future detections here)
switch get(handles.detection_popup, 'Value')
    case 1  % Saliency Points Detector
        handles.settings.detection.id = 'salient_points';
    case 2  % Custom Extraction
        handles.settings.detection.id = 'custom_extraction';
end
...
```

Then, we edit the function *artgui_OutputFcn*:

```
...
% Open the correct detection gui (add future detections here)
if handles.next_pressed
    switch handles.settings.detection.id
        case 'salient_points'
            [data, settings] = ...
                salientpointsdetectiongui(data, settings);
        case 'custom_extraction'
            [data, settings] = ...
                customextractiongui(data, settings);
    end
end
...
```

Now, we can edit the template file **customextractiongui** using the command

```
guide customextractiongui;
```

and add all the components relative to the parameters (which must be saved into the structure *settings*) of our new keypoint extraction method.

Then, we digit into the MATLAB command line

```
edit artexec;
```

to edit the code as follows:

```
...
% Open the correct detection exec (add future detection here)
switch settings.detection.id
    case 'salient_points'
        data = salientpointsdetectionexec(data, settings);
    case 'custom_extraction'
        data = customextractionexec(data, settings);
end
...
```

Lastly, we edit the function `customextractionexec` using the command

```
edit customextractionexec;
```

so that it executes our new custom algorithm.

Adding a Keypoint Description module

Let's assume we want to add a new keypoint description module called **customdescription** to ART.

As a first step we add the new module to the drop-down menu *Description* in the main window of the toolkit (Figure 2). This process is similar to the one used in Adding a Keypoint Extraction module. We use the MATLAB command

```
guide artgui;
```

to modify ART's main window (Figure 16).

From there we can change the properties of the descriptors drop-down menu by adding **Custom Description** to the field *String*.

Then, we edit the code relative to ART's GUI using the command

```
edit artgui;
```

to edit the functions *artgui_OpeningFcn* and *description_popup_Callback*:

```
...
% Get the description (add future descriptions here)
switch get(handles.description_popup, 'Value')
    case 1  % Spin Images
        handles.settings.description.id = 'spin_images';
    case 2  % Custom Description
        handles.settings.description.id = ...
            'custom_description';
end
...
```

After that, we modify all the implemented keypoint extraction modules (for our example we use the Salient Points module). We write

```
edit salientpointsdetectiongui;
```

into MATLAB's command line and we change the function *salientpointsdetectiongui_OutputFcn*:

```
...
% Open the correct description gui...
if handles.next_pressed
    switch handles.settings.description.id
        case 'spin_images'
            [data, settings] = spinimagesgui(data, settings);
        case 'custom_description'
            [data, settings] = ...
                customdescriptiongui(data, settings);
    end
end
...
```

Then, we modify the template file `customdescritpiongui` using the command

```
guide customdescriptiongui;
```

to insert all the graphical components relative to the new module's parameters (which must be saved into the *settings* structure).

After that (we still consider the Salient Points module), we write

```
edit salientpointsdetectionexec;
```

to modify the code as follows:

```
...
% Open the correct description exec...
switch settings.description.id
    case 'spin_images'
        data = spinimagesexec(data, settings);
    case 'custom_description'
        data = customdescriptionexec(data, settings);
end
...
```

As a last step, we edit the template file `customdescriptionexec` using the command

```
edit customdescriptionexec;
```

so that it executes our new custom keypoint description algorithm.

Adding a Pairwise Registration module

Let's assume we want to add a new pairwise registration module called **Custom Pairwise** to ART.

As a first step we add the new module to the drop-down menu *Pairwise Registration* in the main window of the toolkit (Figure 2). This process is similar to the one used in Adding a Keypoint Extraction module. We use the MATLAB command

```
guide argui;
```

to modify ART's main window (Figure 16).

From there we can change the properties of the considered drop-down menu by adding **Custom Pairwise** to the field *String*.

Then, we edit the code relative to ART's GUI using the command

```
edit artgui;
```

to edit the functions *artgui_OpeningFcn* and *pairwise_popup_Callback*:

```
...
% Get the pairwise (add future pairwise here)
switch get(handles.pairwise_popup, 'Value')
    case 1  % Spin Images
        handles.settings.pairwise.id = 'msac_icp';
    case 2  % Custom Pairwise
        handles.settings.pairwise.id = 'custom_pairwise';
end
...
```

After that, we modify all the implemented keypoint description modules (for our example we use the Spin Images module). We write

```
edit spinimagesgui;
```

into MATLAB's command line and we change the function *spinimagesgui_OutputFcn*:

```
...
% Open the correct pairwise gui (add future pairwise here)
    if handles.next_pressed
        switch handles.settings.pairwise.id
            case 'msac_icp'
                [data, settings] = msacicpgui(data, settings);
            case 'custom_pairwise'
                [data, settings] = ...
                    custompairwisegui(data, settings);
    end
end
...
```

Then, we modify the template file `custompairwisegui` using the command

```
guide custompairwisegui;
```

to insert all the graphical components relative to the new module's parameters (which must be saved into the *Settings* structure).

After that (we still consider the Spin Images module), we write

```
edit spinimagesexec;
```

to modify the code as follows:

```
...
% Open the correct pairwise exec (add future pairwise here)
switch settings.pairwise.id
    case 'msac_icp'
        data = msacicpexec(data, settings);
    case 'custom_pairwise'
        data = custompairwiseexec(data, settings);
end
...
```

Lastly, we edit the template file `custompairwiseexec` using the command

```
edit custompairwiseexec;
```

so that it executes our new custom pairwise registration algorithm.

Adding a Multi-view Registration module

Let's assume we want to add a new multi-view registration module called **custommultiview** to ART.

As a first step we add the new module to the drop-down menu *Multiview Registration* in the main window of the toolkit (Figure 2). This process is similar to the one described in Adding a Keypoint Extraction module. We use the MATLAB command

```
guide artgui;
```

to modify ART's main window (Figure 16).

From there we can change the properties of the multi-view registration drop-down menu by adding **Custom Multiview** to the field *String*.

Then, we edit the code relative to ART's GUI using the command

```
edit artgui;
```

to edit the functions *artgui_OpeningFcn* and *multiview_popup_Callback*:

```
...
% Get the multiview (add future multiview here)
switch get(handles.multiview_popup, 'Value')
    case 1  % Pairwise
        handles.settings.multiview.id = ...
            'pairwise_multiview_alignment';
    case 2  % Custom Pairwise
        handles.settings.multiview.id = 'custom_pairwise;
end
...
```

After that, we modify all the implemented pairwise registration modules (for our example we use the MSAC + ICP module). We write

```
edit msacicpgui;
```

into MATLAB's command line and we change the function *msacicpgui_OutputFcn*:

```
...
% Open the correct multiview gui (add future multiview here)
if handles.next_pressed
    switch handles.settings.multiview.id
        case 'pairwise_multiview_alignment'
            [data, settings] = ...
                pairwisemultiviewalignmentgui(data, settings);
        case 'custom_multiview'
            [data, settings] = ...
                custommultiviewgui(data, settings);
    end
end
...
```

Then, we modify the template file **custommultiviewgui** using the command

```
guide custommultiviewgui;
```

to insert all the graphical components relative to the new module's parameters (which must be saved into the *settings* structure).

After that (we still consider the MSAC + ICP module), we write

```
edit msacicpexec;
```

to modify the code as follows

```
...
% Open the correct multiview exec (add future multiview here)
switch settings.multiview.id
    case 'pairwise_multiview_alignment'
        data = ...
            pairwisemultiviewalignmentexec(data, settings);
    case 'custom_pairwise'
        data = custompairwiseexec(data, settings);
end
...
```

As a last step, we edit the template file `custommultiviewexec` using the command

```
edit custommultiviewexec;
```

so that it executes our new custom multi-view registration module.

References

[1] MATLAB. *version 8.3 (R2014a)*. The Mathworks Inc., Natick, Massachusetts, 2014.

[2] Mario Botsch, Stefan Steinberg, Stefan Bischoff, and Leif Kobbelt. Openmesh: A generic and efficient polygon mesh data structure. In *OpenSG Symposium 2002*, 2002.

[3] Gaël Guennebaud, Benoît Jacob, et al. Eigen v3. http://eigen.tuxfamily.org, 2010.

[4] D. Mount and S. Arya. Ann: A library for approximate nearest neighbor searching. `https://www.cs.umd.edu/~mount/ANN/`, January 2010. Version 1.1.2.

[5] Andrea Fusiello. *Visione Computazionale. Tecniche di Ricostruzione Tridimensionale*. Franco Angeli, Milano, 2013.

[6] U. Castellani, A. Fusiello, and V. Murino. Registration of multiple acoustic range views for underwater scene reconstruction. *Computer Vision and Image Understanding*, 87(3):78–89, 2002.

[7] Frank R. Hampel, Elvezio M. Ronchetti, Peter J. Rousseeuw, and Werner A. Stahel. *Robust Statistics: The Approach Based on Influence Functions (Wiley Series in Probability and Statistics)*. Wiley-Interscience, New York, revised edition, April 2005.

[8] U. Castellani, M. Cristani, S. Fantoni, and V. Murino. Sparse points matching by combining 3d mesh saliency with statistical descriptors. *Computer Graphics Forum*, 27(2):643–652, 2008.

[9] Andrew Johnson. *Spin-Images: A Representation for 3-D Surface Matching*. PhD thesis, Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, August 1997.

[10] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, SGP '06, pages 61–70, Aire-la-Ville, Switzerland, Switzerland, 2006. Eurographics Association.

[11] Michael Kazhdan and Hugues Hoppe. Screened poisson surface reconstruction. *ACM Trans. Graph.*, 32(3):29:1–29:13, July 2013.